



University Of Information Technology, Shimla (H.P.)

Department of Computer Science Engineering

Handwritten Digit Recognition System

- Submitted By:
 - Riya Sharma (17242990050)
 - Arpita Sood (17242990019)
- Team Name: Techie Twinz
- Guided By: Er. Shweta Rajput
- Course: B.Tech CSE (4th Semester)

✓ PROJECT DETAILS :-

- Project: Handwritten Digit Recognition using Convolutional Neural Network (CNN)
- Language: Python
- Libraries Used:
 - TensorFlow
 - Keras
 - NumPy
 - Matplotlib
 - Scikit learn
 - Seaborn
- Description: This program trains a CNN model on the MNIST dataset to recognize handwrittendigits (0–9). The model is trained using training images and then tested on unseen images to predict the correct digit.

✓ PYTHON PROGRAM :-

```
# handwritten digit recognition using cnn model

import random
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns

# load the dataset
(x_train,y_train),(x_test,y_test) = keras.datasets.mnist.load_data()

# normalize the data
x_train = x_train / 255.0
x_test = x_test / 255.0

# reshape the data to fit the model
x_train = x_train.reshape(-1,28,28,1)
x_test = x_test.reshape(-1,28,28,1)

# build the model
model = keras.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
```

```

layers.MaxPooling2D((2,2)),

# second layer
layers.Conv2D(64,(3,3),activation='relu'),
layers.MaxPooling2D((2,2)),

# flatten layer
layers.Flatten(),

# dense layers
layers.Dense(64,activation='relu'),

# output layer
layers.Dense(10,activation='softmax')
])

# compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# train the model
history = model.fit(
    x_train,y_train,
    epochs=3,
    validation_data=(x_test,y_test)
)

# evaluate the model
test_loss, test_acc = model.evaluate(x_test,y_test)
print(f'Test accuracy: {test_acc*100:.2f}%')

# confusion matrix
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
cm = confusion_matrix(y_test, y_pred_classes)

# predicting a single image
index = random.randint(0, len(x_test)-1)

img = x_test[index].reshape(1,28,28,1)

prediction = model.predict(img)

```

```
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.show()

print("Predicted Digit:", np.argmax(prediction))
print("Actual Digit:", y_test[index])
```

✓ DEMONSTRATION:-

• IMPORTING LIBRARIES:-

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

- ❖ Used for building and training the CNN model
- ❖ Other libraries:

- **numpy** → numerical operations
- **matplotlib** → display images
- **sklearn** → confusion matrix
- **random** → pick random test image

• LOADING DATASET:

```
(x_train,y_train),(x_test,y_test) = keras.datasets.mnist.load_data()
```

❖ Loads **MNIST dataset**

- 60,000 training images
- 10,000 testing images
- Each image is **28×28 pixels (grayscale)**

❖ Labels (0–9) represent digits

• **DATA PREPROCESSING:**

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

- ❖ Normalization (0–255 → 0–1)
- ❖ Improves training speed and accuracy

```
x_train = x_train.reshape(-1,28,28,1)
x_test = x_test.reshape(-1,28,28,1)
```

❖ CNN needs 4D input:

- samples, height, width, channels
- Here channels = 1 (grayscale)

- **BUILDING CNN MODEL:**

```
model = keras.Sequential([
```

- **LAYER 1: CONVOLUTION:**

```
layers.Conv2D(32,(3,3),activation='relu',
```

- Detects features like edges

- **LAYER 2: POOLING:**

```
layers.MaxPooling2D((2,2)),
```

- Reduces image size

- **LAYER 3 & 4: MORE FEATURES EXTRACTION :**

```
layers.Conv2D(64,(3,3),activation='relu'),
layers.MaxPooling2D((2,2)),
```

- Learns deeper patterns (curves, shapes)

- **LAYER 5: FLATTEN LAYER :**

```
layers.Flatten(),
```

- Converts 2D → 1D

- **LAYER 6 : DENSE LAYER:**

```
layers.Dense(64,activation='relu'),
layers.Dense(10,activation='softmax')
```

- 10 neurons → digits (0–9)

- Softmax → gives probabilities

- **COMPILING MODEL:**

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

- Adam → fast optimizer
- Loss function → for classification

- **TRAINING MODEL:**

```
history = model.fit(  
    x_train,y_train,  
    epochs=3,  
    validation_data=(x_test,y_test)  
)
```

- Model learns patterns from images
- Each epoch = one full pass of data

- **MODEL EVALUATION:**

```
test_loss, test_acc = model.evaluate(x_test,y_test)
```

- Tests model on unseen data

- **CONFUSION MATRIX:**

```
cm = confusion_matrix(y_test, y_pred_classes)
```

- Correct predictions (diagonal)
- Wrong predictions (off-diagonal)

- **PREDICTING RANDOM IMAGE:**

```
index = random.randint(0, len(x_test)-1)  
img = x_test[index].reshape(1,28,28,1)  
prediction = model.predict(img)
```

✓ OUTPUT SCREENSHOTS :-

The image shows a Visual Studio Code editor window with a Python script titled "# handwritten digit recognition using cn.py 3". The script is located at "C:\Users\priya>OneDrive>Documents># handwritten digit recognition using cn.py > ...". The code includes a plot display and print statements for predicted and actual digits.

```
71 plt.show()
72
73 print("Predicted Digit:", np.argmax(prediction))
74 print("Actual Digit:", y_test[index])
75
```

The terminal output shows a warning about TensorFlow GPU support on Windows, followed by training progress for three epochs and a final test accuracy of 98.83%.

```
object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1773152232.207178    7736 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instruct
ions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlo
w with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are insta
lled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/3
1875/1875 ██████████ 25s 12ms/step - accuracy: 0.9548 - loss: 0.1479 - val_accuracy: 0.9845 - val_loss: 0.0484
Epoch 2/3
1875/1875 ██████████ 23s 12ms/step - accuracy: 0.9855 - loss: 0.0471 - val_accuracy: 0.9860 - val_loss: 0.0396
Epoch 3/3
1875/1875 ██████████ 42s 13ms/step - accuracy: 0.9897 - loss: 0.0332 - val_accuracy: 0.9883 - val_loss: 0.0356
313/313 ██████████ 2s 6ms/step - accuracy: 0.9883 - loss: 0.0356
Test accuracy: 98.83%
313/313 ██████████ 2s 5ms/step
1/1 ██████████ 0s 56ms/step
Predicted Digit: 2
Actual Digit: 2
PS C:\Users\priya\OneDrive\Documents\C language\chapter1>
```

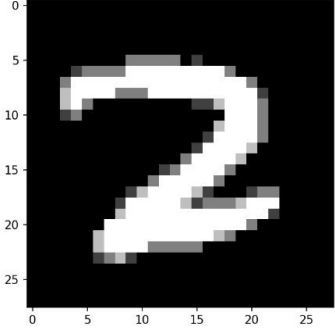
The status bar at the bottom indicates the file is at line 75, column 1, using UTF-8 encoding with CRLF line endings. The editor is configured for Python 3.13 (64-bit) with Go Live and Prettier extensions.

File Edit Selection ... chapter1

handwritten digit recognition using cn.py 3

C: > Users > priya > OneDrive > Documents > # handwritten digit recognition using cn.py > ...

71 plt.show()
72 Figure 1
73
74
75



PROBLEM

C:\Use
ning:
object
super
10000
ions i
To ena
w with
WARNIN
lled,
Epoch
1875/1
loss: 0.1479 - val_accuracy: 0.9845 - val_loss: 0.0484
Epoch 2/3
1875/1875 23s 12ms/step - accuracy: 0.9855 - loss: 0.0471 - val_accuracy: 0.9860 - val_loss: 0.0396
Epoch 3/3
1875/1875 42s 13ms/step - accuracy: 0.9897 - loss: 0.0332 - val_accuracy: 0.9883 - val_loss: 0.0356
313/313 2s 6ms/step - accuracy: 0.9883 - loss: 0.0356
Test accuracy: 98.83%
313/313 2s 5ms/step
1/1 0s 56ms/step

Python Debug Console

ackages\keras\src\layers\convolutional\base_conv.py:113: UserWar
. When using Sequential models, prefer using an 'Input(shape)'
args)
is TensorFlow binary is optimized to use available CPU instruct
AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlo
ve Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are insta
ctML plugin.

Ln 75, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.13 (64-bit) Go Live Prettier

18°C Clear Search 19:49 10-03-2026